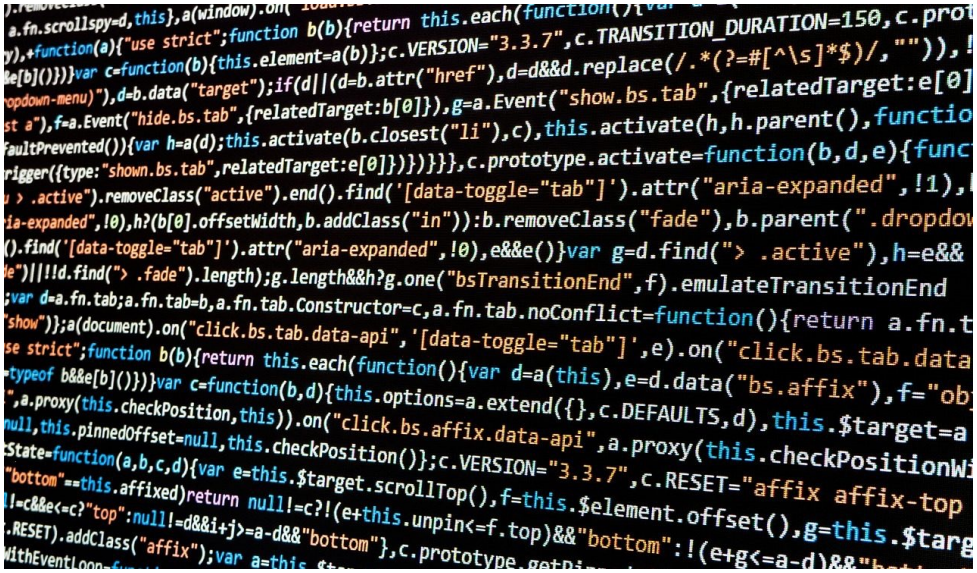
Travail de maturité

Les équations polynomiales : de l'histoire antique aux algorithmes modernes



Cléophée LOEMBA TCHIBOLI

Gymnase de Burier
3M16
Suivi par M. T. BESSON

Octobre 2025

Résumé

Mon travail de maturité, intitulé « Les équations polynomiales : de l'histoire antique aux algorithmes modernes », retrace une partie de l'évolution des équations polynomiales jusqu'à leur traitement à l'aide de programmes informatiques contemporains.

Dans la première partie, j'ai exploré l'histoire des polynômes à travers différentes civilisations et mathématiciens. Les premières traces de l'existence des polynômes remontent à la civilisation babylonienne environ 2000 ans avant J.-C. Ils résolvaient des équations polynomiales du second degré sur des problèmes liés à la pratique du commerce ou à la géométrie. Bien des siècles plus tard, les Grecs comme Euclide ou Diophante d'Alexandrie ont apporté une vision plus symbolique des équations. Au IXe siècle, Al-Khwarizmi a posé les bases de l'algèbre et a introduit une méthode systématique de résolution des équations polynomiales du second degré. Puis, j'ai exploré certaines avancées majeures de la Renaissance avec des mathématiciens comme Ferrari ou Cardan.

Grâce à ces recherches, j'ai pu voir la continuité entre l'histoire antique des polynômes et les raisonnements algébriques modernes.

Dans la seconde partie, j'ai programmé plusieurs algorithmes en langage Python qui effectuent les opérations principales sur les polynômes : addition, soustraction, multiplication, résolution des équations du second degré, factorisation avec le schéma de Horner, dérivation et division. Puis, j'ai assemblé toutes ces fonctions dans un programme complet avec menu pour que l'utilisateur puisse choisir l'opération qu'il souhaite effectuer.

Pour conclure, grâce à ce travail j'ai pu relier l'histoire, les théories mathématiques et l'informatique. J'ai développé une méthode de travail rigoureuse, analytique, logique et j'ai développé ma connaissance du langage Python. De plus, j'ai appris à anticiper les erreurs possibles et structurer mes codes de façon claire et efficace. Mes recherches m'ont donné une vision globale de ce concept mathématique et ont renforcé mon intérêt pour ces domaines d'étude.

Remerciements

Je tiens à exprimer mes remerciements les plus sincères à Monsieur Besson, mon maître répondant qui m'a accompagnée tout au long de ce travail de maturité pour son attention, ses conseils et sa rigueur.

Ses orientations m'ont permis de progresser et d'allier la précision mathématique et informatique dans la réalisation de mon travail.

Je souhaite également remercier l'ensemble du gymnase de Burier pour l'encadrement attentif, encourageant et les outils mis à ma disposition.

Enfin, je remercie ma famille pour son soutien constant et sa présence essentielle durant ces mois de réalisation.

Table des matières

Introduction **7**

1 Notion de polynôme **8**
1.1 L'invention et la découverte des polynômes par les Grecs et les Babyloniens 8
1.2 Les apports d'Al-Khwarizmi au Moyen-Âge 9
1.3 Les polynômes du 3e et 4e degré au XVIe siècle 11

2 Programmation des opérations sur les polynômes **12**
2.1 Création d'algorithmes pour les opérations fondamentales sur les polynômes 12
2.1.1 Addition de polynômes 12
2.1.2 Soustraction de polynômes 13
2.1.3 Multiplication de polynômes 14
2.1.4 Le schéma de Horner 15
2.1.5 Équation polynomiale du second degré 16
2.1.6 Multiplication d'un polynôme par un nombre 18
2.1.7 Division d'un polynôme par un nombre 18
2.1.8 Dérivation de polynômes 18
2.2 Combinaison des scripts 19

Conclusion **23**

Bibliographie **25**

Table des illustrations **26**

Annexes

Introduction

Un polynôme est une expression mathématique composée uniquement de produits et de sommes de constantes et de variables. Ce mot est formé de « poly » qui signifie plusieurs en grec et de «nomos» le terme. Le polynôme est une somme de termes. Les polynômes s'écrivent sous la forme : $a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$.

Les équations polynomiales sont essentielles et centrales en mathématiques depuis le début de la civilisation humaine. L'objectif de la résolution d'une telle équation est de trouver l'ensemble des valeurs de l'inconnue, les solutions de l'équation, pour lesquelles l'équation polynomiale est vraie. Les mathématiciens se sont intéressés à la résolution d'équations polynomiales à travers différents domaines comme la géométrie, la physique ou encore l'économie. [1]

Historiquement, les équations polynomiales ont fait leur apparition chez le peuple babylonien, vers 2000 av. J.-C., sans véritables modèles algébriques. D'autres civilisations, comme les Grecs, se sont intéressées à la question de l'équation polynomiale et ont chacune contribué à son développement. Cette évolution a permis à l'humanité à travers les siècles de résoudre une multitude de problèmes mathématiques tels que le calcul de trajectoires ou la résolution d'équations géométriques et physiques.

Grâce aux recherches que j'ai effectuées, ce travail m'a permis d'explorer l'évolution des polynômes et leur importance dans les mathématiques. J'ai décidé de consacrer mon travail de maturité à ce thème en raison de mon intérêt pour les mathématiques ainsi que pour l'informatique. La combinaison de ces deux centres d'intérêt m'a permis d'approfondir l'étude d'un sujet qui ne m'était pas familier. Mon intérêt s'est développé en cours d'informatique où j'ai pu apprendre les notions nécessaires à l'écriture de programmes et en mathématiques où j'ai acquis des connaissances me permettant d'effectuer différentes opérations sur les polynômes.

Dans ce travail de maturité, j'ai exploré les origines et le développement des équations polynomiales et je présente des programmes informatiques capables de résoudre ces équations et d'effectuer des opérations sur les polynômes.

La problématique a été la suivante : Comment la représentation et la manipulation des équations polynômiales ont-elles évolué au fil du temps et comment peut-on les modéliser aujourd'hui à l'aide d'algorithmes ?

1. Les équations polynomiales sont encore utilisées, notamment en économie pour modéliser les fluctuations des marchés financiers.

1 – Notion de polynôme

1.1 L'invention et la découverte des polynômes par les Grecs et les Babyloniens

Avant d'aborder la partie informatique, j'ai souhaité donner un contexte historique aux équations polynomiales afin de comprendre leurs origines et la façon dont ce concept est apparu.

Les premières traces de l'existence des polynômes remontent à l'Antiquité plus précisément à Babylone, une cité antique de Mésopotamie. C'est dans cette région que de nombreuses inventions indispensables au développement des sociétés qui ont suivi ont émergé. Par exemple, dans le domaine du droit c'est à cet endroit que les lois les plus anciennes ont été écrites grâce au système d'écriture sur des tablettes d'argile (le cunéiforme) et que l'architecture a été développée.

Les mathématiques étaient utilisées afin de distribuer les richesses, construire du mobilier, ou bien encore calculer des coûts. D'après de nombreuses traces retrouvées, notamment sur des tablettes d'argiles, les Babyloniens savaient résoudre des équations du second degré $ax^2 + bx + c = 0$, une forme d'équation utilisée pour résoudre des problèmes concrets. Les Babyloniens n'utilisaient certes pas encore la notation symbolique que nous utilisons aujourd'hui, pour autant leurs méthodes montraient les premiers raisonnements algébriques et donc l'une des origines des polynômes.

Quelques siècles plus tard, dans le monde grec antique, plus précisement à Alexandrie, les mathématiciens ont repris et approfondi les notions découvertes par les Babyloniens. Euclide, un mathématicien grec du IIIe siècle av. J.-C. a posé les bases de la géométrie dans son traité mathématique nommés «Les Eléments» où il démontre des propriétés fondamentales sur les figures, les rapports et les proportions. Son approche géométrique a permis de représenter les équations polynomiales à l'aide de figures.



Portrait d'Euclide, mathématicien grec antique

Bien des siècles après, Diophante d'Alexandrie fit un pas considérable vers l'algèbre moderne. Diophante d'Alexandrie a été un mathématicien grec ayant vécu à Alexandrie entre le I^{e} siècle et le IIIe siècle. Il est considéré comme le père de l'algèbre. Dans sa célèbre série de livres «Arithmetica», il explique comment écrire des équations mathématiques à l'aide de symboles ainsi que comment résoudre des équations polynomiales. Il a également contribué au développement des équations diophantiennes, des équations polynomiales ayant une ou plusieurs inconnues dont les solutions cherchées sont des nombres entiers ou rationnels et dont les coefficients sont entiers.



Portrait de Diophante d'Alexandrie, mathématicien grec

De ce fait, de Babylone à la Grèce, les mathématiques ont eu une approche pratique et géométrique ayant évolué vers un raisonnement symbolique et plus abstrait. Cela marque les premières étapes de la découverte et de la compréhension des polynômes.

1.2 Les apports d'Al-Khwarizmi au Moyen-Âge

Au IXe siècle, le mathématicien perse Muḥammad ibn Mūsā al-Khwārizmī rédigea un texte fondateur de l'algèbre nommé «Kitāb al-mukhtaṣar fī ḥisāb al-jabr wa-l-muqābala» qui signifie «L'abrégé du calcul par la restauration et la réduction». L'algèbre est un héritage laissé par les civilisations grecques et orientales qui plus tard a été développé au Moyen Âge. Al-Khwarizmi a, dans cet ouvrage, posé les fondations de l'algèbre et a étudié de manière systématique la résolution d'équations polynomiales du premier et second degré. Il a introduit une méthode algébrique encore inconnue à l'époque et a mis en valeur l'utilisation du zéro.
À l'époque, les mathématiques n'étaient pas fondées sur la symbolique utilisée de nos jours x,y etc., mais étaient fondées sur des descriptions verbales et géométriques. Par exemple, une équation géométrique comme :

$$x^2 + 10x = 39$$

était représentée en termes concrets : «un carré et dix racines (l'inconnue x car elle désigne la valeur à multiplier par elle-même pour mettre au carré) égalent 39».
Pour résoudre une équation quadratique, Al-Khwarizmi utilisait une approche géométrique.

1. Représentation de la figure Il faut d'abord dessiner un carré de côté x pour représenter x^2. Puis, on place deux rectangles de dimensions $x \cdot 5$ le long de deux côtés du carré car $10x = 2 \cdot 5x$. La figure ressemble désormais à un grand carré presque complet mais il manque un carré dans le coin pour que celui-ci soit parfait.

2. Complétion du carré Le côté du carré manquant vaut la moitié de 10 soit 5, donc son aire est $5^2 = 25$. En ajoutant ce carré manquant, la figure est un carré parfait de côté $x + 5$. Cependant, pour conserver l'égalité il faut ajouter 25 l'autre côté de l'équation.

$$x^2 + 10x + 25 = 39 + 25$$

3. Simplification Le côté gauche de l'équation peut se réécrire tel que :

$$x^2 + 10x + 25 = (x + 5)^2$$

et le côté droit devient 64 car égal à $39 + 25$
On obtient donc :

$$(x + 5)^2 = 64$$

4. Résolution On utilise la racine carrée des deux côtés : $x + 5 = \pm 8$ On soustrait 5 des deux côtés :

$$x = -5 \pm 8$$

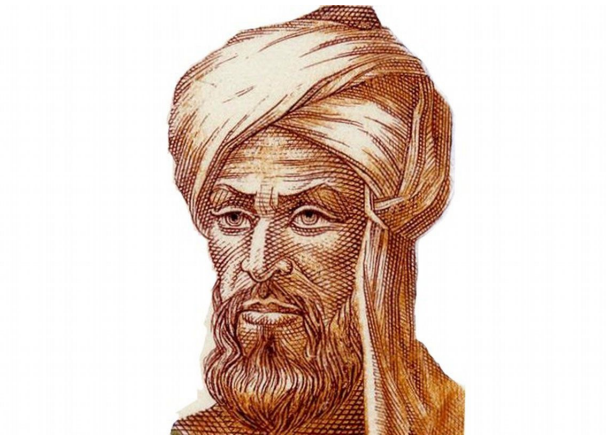
Les solutions sont :

$$x = 3$$

ou

$$x = -13$$

Or, une surface ne peut être que positive ou nulle donc seule la solution positive était gardée.
Le nom Al-Khwarizmi a donné le mot «algorithme» un mot central dans mon travail de maturité puisque les algorithmes modernes prolongent directement sa logique de résolution systématique des équations.



Portrait d'Al-Khwarizmi, mathématicien perse

1.3 Les polynômes du 3e et 4e degré au XVIe siècle

Au XVIe siècle, des mathématiciens européens se sont intéressés aux équations polynomiales de degrés supérieurs, c'est-à-dire aux équations cubiques (3e degré) et quartiques (4e degré). Ces équations se sont avérées beaucoup plus complexes à résoudre que celles étudiées précédemment au cours de l'histoire.

Parmi les principaux mathématiciens de cette époque, nous pouvons citer :

Scipione del Ferro (1465 - 1526) qui est un mathématicien italien célèbre pour avoir trouvé en premier la méthode de résolution d'équations du troisième degré sans terme quadratique. Par exemple, une équation du troisième degré s'écrit : $ax^3 + bx^2 + cx + d = 0$; sans terme quadratique elle s'écrit : $x^3 + cx + d = 0$

Niccolò Tartaglia (1499-1557) qui est un mathématicien italien et qui a approfondi les travaux de Scipione del Ferro a découvert des formules générales comme celle pour trouver la solution réelle de l'équation cubique. Pour une équation du type : $x^3+px = q$, la solution réelle peut s'écrire :

$$x = \sqrt[3]{\frac{q}{2} + \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}} + \sqrt[3]{\frac{q}{2} - \sqrt{\left(\frac{q}{2}\right)^2 + \left(\frac{p}{3}\right)^3}}$$

Ludovico Ferrari (1522-1565) qui est un mathématicien italien qui a élaboré des méthodes pour résoudre des équations quartiques. Il a notamment travaillé avec Cardan et ses travaux ont été publiés par celui-ci dans «Ars Magna» en 1545.

Toutes leurs découvertes ont permis de déterminer des formules générales marquant un véritable tournant dans l'histoire des équations polynomiales. Les équations polynomiales n'étaient plus simplement des modèles géométriques mais de véritables objets mathématiques et cela a permis d'ouvrir la voie à l'algèbre moderne.

Cependant, pour les équations polynomiales de degré cinq et plus, il a été démontré qu'aucune formule générale n'existe ce qui montre la complexité croissante et la nécessité de trouver des méthodes alternatives pour les résoudre. Cette complexité a conduit au développement de méthodes numériques et algorithmiques afin de calculer les racines de polynômes de degré élevé et de les manipuler avec l'informatique.
L'étude des polynômes à ce stade a permis de mieux comprendre des concepts fondamentaux tels que les racines multiples, la factorisation, ainsi que les coefficients et leur rôle.

Ces avancées historiques sont à la base des algorithmes que nous allons développer dans ce travail. Grâce à la programmation, il est désormais possible de résoudre des équations polynomiales et d'effectuer des opérations complexes sur des polynômes de degré élevé dont la résolution manuelle serait extrêmement difficile.
Cette transition de la théorie historique à la pratique informatique illustre l'évolution et l'importance des polynômes dans les mathématiques et les sciences.

2 – Programmation des opérations sur les polynômes

2.1 Création d'algorithmes pour les opérations fondamentales sur les polynômes

Avant de commencer à travailler à coder un script, il a d'abord fallu réfléchir à ce que je voulais réellement représenter dans ce projet. Mon objectif était d'écrire un script Python capable d'effectuer des opérations sur les polynômes : addition, soustraction, multiplication, division, division selon le schéma de Horner, dérivation.

Il a donc fallu réfléchir à la manière de représenter chaque opération. J'ai alors écrit un script distinct pour chacune d'elles.
Il fallait créer du code qui permettrait à l'utilisateur d'entrer des valeurs de coefficients polynômiaux. Je me suis donc interrogée sur la façon de représenter ces coefficients dans un script. Quelle structure de données était la plus adaptée ? Pendant cette phase, je n'ai eu que des réflexions théoriques qui m'ont ensuite permis d'appliquer ma réflexion à mon code.

Il a fallu représenter les polynômes à l'aide d'un code python et j'ai décidé d'utiliser des listes. Celles-ci permettent de stocker facilement les coefficients et de pouvoir faire des opérations à partir de ces données. Avec cette solution, les monômes composant le polynôme doivent être rangés dans un ordre précis. Le plus logique me paraissait être l'ordre décroissant des degrés. Ainsi, un polynôme de degré 4 devra avoir son coefficient de degré 4, 3, 2, 1 puis 0. Par exemple : $2x^4+3x^3-7x^2-2x+1$ et non pas $1-7x^2+3x^3+2x^4-2x$. Cette représentation s'est avérée pertinente pour les opérations polynomiales, mais exige une certaine rigueur de la part de l'utilisateur.

Par la suite, j'ai réfléchi à faire en sorte que l'utilisateur entre la liste des coefficients d'un polynôme et ensuite que le script exécute certaines opérations. Il y a donc un input de valeur, puis les fonctions `split` et `map(int)` qui convertissent les chaînes de caractères en entiers directement dans une liste. Cette étape m'a demandé plusieurs essais et tests avant de trouver une méthode fluide et logique.

2.1.1 Addition de polynômes

Une fois que la structure était posée, j'ai commencé par l'addition de polynômes qui me semblait la plus évidente.

L'une des premières erreurs que j'ai pu rencontrer était d'additionner les polynômes entrés sans prendre en compte le degré du polynôme. Pour résoudre ce problème, j'ai pensé que si le premier polynôme est plus petit que le deuxième, il y aura une inversion.

C'est à ce moment-là que j'ai réalisé qu'il fallait réellement réfléchir comme une machine qui applique exactement ce qu'on lui demande. Il faut que tout soit prévu même dans les cas les plus rares auxquels nous ne penserions pas du premier coup comme dans le cas du polynôme nul.

J'ai donc écrit un script qui compare la longueur de polynômes. Si le premier entré est plus petit que le deuxième, alors ils s'inversent. Il est important que le plus long polynôme soit placé en premier.

Ensuite, il faut que le code parcourt les deux listes et additionne les termes de même degré à l'aide d'une boucle. Celle-ci s'arrête à la longueur du plus petit polynôme afin que les termes restants du polynôme le plus grand soient inchangés.
Voici la fonction Python correspondante :

```
1 def addition(p1, p2):
2     if len(p2) > len(p1):
3         p1, p2 = p2, p1  # On s'assure que p1 est le plus long
4     p1 = p1.copy()  # Pour ne pas modifier la liste originale
5     for i in range(len(p2)):
6         p1[i] += p2[i]
7     return p1
8
```

Listing 2.1 – Addition de polynômes

2.1.2 Soustraction de polynômes

Après avoir élaboré la fonction d'addition, j'ai ensuite fait celle de la soustraction, elle me paraissait au premier abord très proche de celle de l'addition.

J'ai donc d'abord réfléchi à la façon dont j'allais m'y prendre. Pour que la soustraction soit possible il est important de considérer la longueur des polynômes. Il fallait donc trouver la longueur maximale de l'un des polynômes si les deux ne sont pas du même degré.

Pour cela j'ai donc pensé la solution qui consiste à compléter le polynôme le plus petit par des coefficients nuls. Afin que les deux polynômes soient de la même longueur si besoin tout en gardant leurs valeurs. Cela revient à compléter le polynôme par des termes de degré supérieur dont les coefficients sont nuls, ce qui est mathématiquement correct.

Puis afin de réaliser la soustraction sur chaque coefficient j'ai utilisé une boucle `while` qui parcourt les deux listes alignées. À chaque itération, la différence entre les coefficients appartenant à la même position est ajoutée à une nouvelle liste «résultat», qui contiendra à la fin les coefficients du résultat de la soustraction.

```python
def soustraction(p1, p2):
    longueur = max(len(p1), len(p2)) #calcul de la longueur maximale car on cherche la longueur la plus grande entre les deux listes
    # permet de gérer les cas où les deux polynômes n'ont pas le même nombre de coefficients (un plus long que l'autre)

    # ajouter des zéros à gauche pour que les deux listes aient la même longueur
    p1_entier = [0] * (longueur - len(p1)) + p1
    p2_entier = [0] * (longueur - len(p2)) + p2

    resultat = []
    position = 0
    while position < longueur:
        resultat.append(p1_entier[position] - p2_entier[position]) # ajouter un élément à la fin d'une liste
        position += 1
    return resultat
```

Listing 2.2 – Soustraction de polynômes

2.1.3 Multiplication de polynômes

Avant de réfléchir à l'écriture d'un script permettant de multiplier deux polynômes entre eux j'ai tout d'abord eu une réflexion sur la méthode utilisée manuellement. Multiplier deux polynômes revient à distribuer chaque terme du premier polynôme à tous les termes du second, puis à réunir les termes de même degré. Par exemple si on multiplie $3x - 4$ par $2x^2-3x+3$, on distribue d'abord chaque terme du premier polynôme avec le deuxième.

Donc :

$$3x \cdot 2x^2 + 3x \cdot (-3x) + 3x \cdot 3$$

Puis :

$$-4 \cdot (2x^2 - 3x + 3)$$

Ce qui nous donne :

$$6x^3 - 9x^2 + 9x - 8x^2 + 12x - 12$$

On additionne ensuite les coefficients de degré identiques :

$$6x^3 - 9x^2 - 8x^2 + 9x + 12x - 12$$

Soit :

$$6x^3 - 17x^2 + 21x - 12$$

Après avoir représenté la procédure manuelle, il a fallu réfléchir à la façon de la représenter sous forme de code. Le but était d'imiter ce processus de façon algorithmique. Les deux polynômes seraient donc sous forme de listes avec des coefficients classés du degré le plus haut au plus bas. J'ai commencé par créer une fonction polynôme prenant en entrée les listes qui représentent les coefficients des deux polynômes à multiplier. Puis j'ai créé

une liste remplie de zéro qui a pour longueur le nombre maximal de termes du nouveau polynôme.

En utilisant une double boucle, la première avec `i` parcourt chaque coefficient du premier polynôme tandis que la seconde boucle, imbriquée à la première, avec `l` parcourt chaque coefficient du second polynôme. À chaque itération, le coefficient du premier polynôme $p1(i)$ est multiplié par celui du second $p2(l)$. Le résultat de chaque multiplication effectuée est ensuite placé dans la case correspondant au bon degré. Cela rendu possible grâce à l'addition des indices `i+l` qui donne le degré des monômes obtenus. Il est important de rappeler qu'en mathématiques lorsqu'il y a une multiplication telle que $x^a x^b$ le résultat est x^{a+b}

Avec d'autres mots, nous pouvons dire que `i` représente la position du coefficient dans le premier polynôme, donc le degré et que `l` la position du coefficient dans le second. En additionnant `i` et `l` on trouve le degré du nouveau coefficient. Par exemple, si on multiplie un terme de degré 1 dans le premier polynôme avec un terme de degré 2 dans le deuxième, on obtient un terme de degré $1+2=3$. Donc il ira à la position 3 dans la liste du résultat. Avec cela, les produits sont placés correctement dans la liste résultat comme ce que l'on fait à la main avant de les additionner. Cependant, il faut aussi implémenter l'addition coefficients.

Le variable `resultat` contiendra donc la somme des termes de même degré. Grâce à cette réflexion, tous les produits sont mis au bon endroit et additionnés correctement. Comme sur le papier, on aligne les termes par degré et on les additionne pour obtenir le polynôme résultat.

```
1  def multiplication(p1, p2):
2      #inversion pour mettre les listes dans l'ordre croissant
3      p1 = p1[::-1]
4      p2 = p2[::-1]
5      resultat = [0] * (len(p1) + len(p2) - 1)  #Résultat est une liste,
       multiplication de deux polynomes a une longeur
6      for i in range(len(p1)):
7          for l in range(len(p2)):
8              resultat[i + l] += p1[i] * p2[l]  # Multiplication des
       coefficients
9      return resultat[::-1] #On réinverse
10
```

Listing 2.3 – Multiplication de polynômes

2.1.4 Le schéma de Horner

Le schéma de Horner qui permet de factoriser un polynôme à l'aide d'une division euclidienne est une façon spéciale de noter la division par $(x-a)$.[1] J'ai voulu créer un programme en python qui permet de factoriser des polynômes de tous les degrés pos-

1. Le schéma de Horner a été présenté pour la première fois par William George Horner, un mathématicien britannique, en 1819.

sibles. Afin d'y parvenir j'ai tout d'abord réfléchi à la méthode et au processus qu'utilise ce schéma. J'ai posé un polynôme du type :

$$a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$$

et réalisé le schéma de Horner sur feuille afin de comprendre le mécanisme. Après avoir compris le mécanisme de cet algorithme, il m'a fallu convertir l'approche que j'avais en une implémentation claire et efficace.

J'ai créé une fonction car le schéma de Horner est un processus répétitif.

La fonction polynôme consiste à prendre en entrée la liste des coefficients du polynôme et la valeur à substituer. Elle initialise le quotient `q` avec le premier coefficient de `p1` et applique la relation de récurrence du schéma de Horner. Puis, elle retourne les coefficients du quotient et le reste de la division.

Une fois l'écriture algorithmique écrite, il a fallu tester le programme avec des polynômes pour s'assurer que le résultat était correct et vérifier que la logique était bien respectée.

```
1 def horner(p1, diviseur):
2     q = [p1[0]]  # Initialisation du premier coefficient
3     for i in range(1, len(p1)):
4         q.append(q[-1] * diviseur + p1[i])  # Application du schéma d'
    Horner
5     return q[:-1], q[-1] # Retourne les coefficients du quotient et le
    reste
6
```

Listing 2.4 – Le schéma de Horner

2.1.5 Équation polynomiale du second degré

Pour résoudre une équation polynomiale du second degré sous la forme

$$ax^2 + bx + c$$

, j'ai tout d'abord réalisé des opérations à la main pour me représenter les étapes par lesquelles on passe. J'ai du implémenter une formule mathématique : le discriminant. J'ai alors créé une fonction nommée `solvequadratic` qui prend en paramètres `a, b et c`. La fonction va calculer le discriminant : $\Delta = b^2 - 4ac$ et retourner les solutions selon les cas possibles, en l'occurrence il y a deux solutions réelles si Δ est positif, une seule si Δ est nul ou aucune si Δ est négatif.

Le script était alors comme ceci :

```python
def resolution_quadratique(a, b, c):
    delta = b*2 - 4a*c # Calcul du discriminant

    if delta > 0:
        solutions = [(-b + delta**0.5) / (2*a), (-b - delta**0.5) / (2*a
    )]
    elif delta == 0:
        solutions = [-b / (2*a)]
    else:
        solutions = ["Aucune solution réelle"]
    return solutions
```

Listing 2.5 – Équation polynomiale du second degré

En revanche, il est souvent possible que Δ soit négatif. J'ai donc voulu modifier mon code pour qu'il puisse retourner des valeurs imaginaires si Δ était inférieur à 0. J'ai donc séparé la solution en deux parties pour qu'elle nous renvoie une partie réelle et une partie imaginaire.

La partie réelle est calculée par moins $-\frac{b}{2a}$ et la partie imaginaire est calculée par $\frac{\sqrt{|\Delta|}}{2a}$. J'ai combiné ces deux parties pour former les solutions complexes et grâce à cela tous les cas possibles d'une équation quadratique peuvent être représentés : il y a deux solutions réelles distinctes si le discriminant est positif, une solution réelle si le discriminant est égal à zéro et deux solutions complexes si le discriminant est plus petit que zéro.

Grâce à cette approche manuelle, j'ai bien compris le rôle du discriminant et la manière dont il détermine la nature des solutions. Cette modification de mon script a permis qu'il puisse être utilisé pour résoudre tout type d'équation quadratique, qu'elle ait des solutions réelles ou imaginaires.

```python
def resolution_quadratique(a, b, c):
    # Calcul du discriminant
    delta = b**2 - 4*a*c

    if delta > 0:
        # Deux solutions réelles distinctes
        solutions = [(-b + delta**0.5) / (2*a), (-b - delta**0.5) / (2*a
    )]
    elif delta == 0:
        # Une solution réelle
        solutions = [-b / (2*a)]
    else:
        # Solutions complexes
        partie_reelle = -b / (2*a)
        partie_imaginaire = (abs(delta)**0.5) / (2*a)
        solutions = [complex(partie_reelle, partie_imaginaire), complex(
    partie_reelle, -partie_imaginaire)]
    return solutions
```

Listing 2.6 – Équation polynomiale du second degré 2.0

2.1.6 Multiplication d'un polynôme par un nombre

Afin d'écrire un script réalisant l'opération de multiplication d'un polynôme par un nombre, j'ai choisi d'utiliser la structure de mes autres fonctions avec une liste de coefficients représentant le polynôme.

J'ai donc créé une fonction qui parcourt chaque coefficient et le multiplie par le nombre `k`. Le résultat est alors une nouvelle liste qui contient les nouveaux coefficients.

Cette façon de procéder est efficace car elle applique directement les propriétés algébriques de la multiplication des polynômes.

```
1 def multiplication_par_nombre(p1, k):
2     return[coef* k for coef in p1]
```

Listing 2.7 – Multiplication d'un polynôme par un nombre

2.1.7 Division d'un polynôme par un nombre

Pour l'opération de division par un nombre, j'ai utilisé la même logique que celle utilisée pour la multiplication par un nombre `k`. J'ai représenté le polynôme par une liste et traité chaque coefficient séparément.

J'ai d'abord créé la fonction `divisionparnombre(p1, k)` divisant chaque coefficient du polynôme par le nombre `k`.
De plus, j'ai ajouté un paramètre essentiel qui signale une erreur s'il y a une division par 0, ce qui est interdit.

```
1 def division_par_nombre(p1, k):
2     if k == 0:
3         print("La division par zéro est impossible.")
4         return None
5     return [coef/ k for coef in p1]
```

Listing 2.8 – Division d'un polynôme par un nombre

2.1.8 Dérivation de polynômes

Après avoir créé différents scripts réalisant des opérations sur les polynômes, j'ai voulu, afin de compléter cette liste, implémenter la dérivation. Cette opération relie la logique mathématique à son interprétation algorithmique.

J'ai d'abord réfléchi à la signification mathématique de la dérivation d'un polynôme et la façon de dériver. Dériver un polynôme revient à multiplier chacun de ses coefficients par le degré de la variable correspondante puis, à réduire ce degré de 1. Par exemple pour le polynôme

$$P(x) = 2x^3 + 5x^2 - 2x + 1$$

on obtient :

$$P'(x) = 6x^2 + 10x - 2$$

Après avoir réalisé plusieurs dérivations manuelles de polynômes, j'ai remarqué que le coefficient constant d'une équation polynomiale doit disparaître car la dérivée d'une constante vaut 0. La boucle doit s'arrêter avant le dernier coefficient.

Après avoir analysé et trouvé la logique j'ai réfléchi à la structure du code. Comme pour toutes les autres fonctions, les coefficients sont rangés du degré le plus élevé au plus faible. La fonction dérivation parcourt tous les coefficients sauf le dernier et multiplie chacun d'eux par son degré.

J'ai créé une liste vide « dérivée » qui se remplit au fur et à mesure avec les coefficients de la dérivée. Le degré initial du polynôme correspond un nombre total de coefficient moins 1. La boucle `for` parcourt tous les éléments, sauf le dernier et applique la formule du calcul de la dérivée à chaque coefficient.

```
#Fonction de dérivation d'un polynôme
def derivation (p1):
    #Le degré du polynôme = nombre de coefficients - 1
    degre = len(p1)-1
    derivee = [] # La liste est vide puis contiendra par la suite les
   coefficents de la dérivée

    #Se fera sur chaque coefficient sauf la constante
    for i in range(len(p1)-1):
    #Ajout à la liste des coefficients de la dérivée
        derivee.append(p1[i]* ( degre - i ))
    return derivee # Renvoie la liste des coefficients dérivés
```

Listing 2.9 – Dérivation de polynôme

2.2 Combinaison des scripts

Après avoir indépendamment écrit du code permettant de réaliser différentes opérations mathématiques, je voulais pouvoir concevoir un assemblage des différentes parties. J'ai donc suivi une démarche progressive et structurée afin d'atteindre cet objectif.

Mon idée était de créer un programme capable de traiter les polynômes de manière concrète en regroupant les fonctionnalités que j'avais écrites séparément au début de ma réflexion. J'ai pris le temps d'analyser chacun de mes programmes et de trouver un lien commun entre chacun d'eux. Il fallait relier le programme qui résolvait un polynôme du second degré avec la méthode Horner à ceux permettaient d'additionner ou de soustraire des polynômes, etc. J'en suis venu à écrire une liste des éléments indispensables à tous mes programmes, ainsi que leurs points communs : l'entrée des données, leur traitement mathématiques et l'affichage du résultat.

C'est cette observation qui m'a conduit à créer un menu principal qui servant de points d'accès unique. J'ai opté pour que l'utilisateur choisisse ce qu'il souhaite faire dès le lancement du programme. Ainsi, le programme pose des questions pour déterminer l'objectif

de l'utilisateur puis appelle le bloc de code correspondant. J'ai dû réfléchir à la façon de relier mes fonctions entre elles sans qu'elles ne se mélangent.

J'ai donc structuré mes algorithmes sous forme de fonctions indépendantes que je pouvais appeler au moment voulu. Cela m'a permis de rendre le code plus clair et surtout réutilisable. Cette partie de mon travail donc été comme un puzzle. Il a fallu organiser de manière logique, placer les codes au bon endroit et veiller à ce que tout reste cohérent et facile à comprendre.

```
def addition(p1, p2):
    if len(p2) > len(p1):
        p1, p2 = p2, p1  # On s'assure que p1 est le plus long
    p1 = p1.copy()  # Pour ne pas modifier la liste originale
    for i in range(len(p2)):
        p1[i] += p2[i]
    return p1

def soustraction(p1, p2):
    longueur = max(len(p1), len(p2)) #calcul de la longueur maximale car on cherche la longueur la plus grande entre les deux listes
    # permet de gérer les cas où les deux polynômes n'ont pas le même nombre de coefficients (un plus long que l'autre)

    # ajouter des zéros à gauche pour que les deux listes aient la même longueur
    p1_entier = [0] * (longueur - len(p1)) + p1
    p2_entier = [0] * (longueur - len(p2)) + p2

    resultat = []
    position = 0
    while position < longueur:
        resultat.append(p1_entier[position] - p2_entier[position]) # ajouter un élément à la fin d'une liste
        position += 1
    return resultat


def multiplication(p1, p2):
    #inversion pour mettre les listes dans l'ordre croissant
    p1 = p1[::-1]
    p2 = p2[::-1]
    resultat = [0] * (len(p1) + len(p2) - 1)  #Résultat est une liste, multiplication de deux polynomes a une longeur
    for i in range(len(p1)):
        for l in range(len(p2)):
            resultat[i + l] += p1[i] * p2[l]  # Multiplication des coefficients
    return resultat[::-1] #On réinverse

def resolution_quadratique(a, b, c):
    # Calcul du discriminant
    delta = b**2 - 4*a*c

    if delta > 0:
        # Deux solutions réelles distinctes
        solutions = [(-b + delta**0.5) / (2*a), (-b - delta**0.5) / (2*a
```

```
    )]
42      elif delta == 0:
43          # Une solution réelle
44          solutions = [-b / (2*a)]
45      else:
46          # Solutions complexes
47          partie_reelle = -b / (2*a)
48          partie_imaginaire = (abs(delta)**0.5) / (2*a)
49          solutions = [complex(partie_reelle, partie_imaginaire), complex(
    partie_reelle, -partie_imaginaire)]
50      return solutions
51
52  def horner(p1, diviseur):
53      q = [p1[0]]  # Initialisation du premier coefficient
54      for i in range(1, len(p1)):
55          q.append(q[-1] * diviseur + p1[i])  # Application du schéma d'
    Horner
56      return q[:-1], q[-1]  # Retourne les coefficients du quotient et le
    reste
57
58  def multiplication_par_nombre(p1, k):
59      return[coef* k for coef in p1]
60
61  def division_par_nombre(p1, k):
62      if k == 0:
63          print("La division par zéro est impossible.")
64          return None
65      return [coef/ k for coef in p1]
66
67  def derivation (p1):
68      degre = len(p1)-1
69      derivee = []
70      for i in range(len(p1)-1):
71          derivee.append(p1[i]* ( degre - i ))
72      return derivee
73
74
75  print("Que souhaitez-vous faire ?")
76  print("1. Addition de deux polynômes")
77  print("2. Soustraction de deux polynômes")
78  print("3. Multiplication de deux polynômes")
79  print("4. Résolution d'une équation polynomiale de degré 2")
80  print("5. Division d'un polynôme par le schéma de Horner")
81  print("6. Multiplication d'un polynôme par un nombre")
82  print("7. Division d'un polynôme par un nombre")
83  print("8. Dérivation d'un polynôme")
84
85  choix = input("Entrez le numéro de l'opération choisie: ")
86
87  if choix in ["1", "2", "3"]:
88      p1 = list(map(int, input("Entrez les coefficients du premier polynô
    me (du plus grand au plus petit) : ").split()))
89      p2 = list(map(int, input("Entrez les coefficients du deuxième polynô
    me (du plus grand au plus petit) : ").split()))
90
91      if choix == "1":
```

```
92          print("Résultat de l'addition :", addition(p1, p2))
93      elif choix == "2":
94          print("Résultat de la soustraction :", soustraction(p1, p2))
95      elif choix == "3":
96          print("Résultat de la multiplication :", multiplication(p1, p2))
97
98  elif choix == "4":
99      p = list(map(int, input("Entrez les coefficients du polynôme (du
        plus grand au plus petit) : ").split())) #p est une liste de
        coefficient
100     if len(p) != 3: #Un polynôme de degré 2 a forcément 3 coefficients.
        S'il n'est pas égal à 3, il y a une erreur à annoncer.
101         print("Un polynôme de degré 2 doit avoir exactement 3
        coefficients.")
102     else:
103         a, b, c = p
104         solutions = resolution_quadratique(a, b, c)
105         print("Solution(s) :", solutions)
106
107 elif choix == "5":
108     p = list(map(int, input("Entrez les coefficients du polynôme (du
        plus grand au plus petit) : ").split()))
109     diviseur = int(input("Entrez la valeur de a dans (x - a) : "))
110     quotient, reste = horner(p, diviseur)
111     print("Quotient :", quotient)
112     print("Reste :", reste)
113
114 elif choix == "6":
115     p1 = list(map(int, input("Entrez les coefficients du premier polynô
        me (du plus grand au plus petit) : ").split()))
116     k = float(input("Entrez le nombre k :"))
117     print("Résultat de la multiplication par k :",
        multiplication_par_nombre(p1, k))
118
119 elif choix =="7":
120     p1 = list(map(int, input("Entrez les coefficients du premier polynô
        me (du plus grand au plus petit) : ").split()))
121     k = float(input("Entrez le nombre k :"))
122     print("Résultat de la division par k :", division_par_nombre(p1, k))
123
124 elif choix == "8":
125     p1 = list(map(int, input("Entrez les coefficients du premier polynô
        me (du plus grand au plus petit) : ").split()))
126     print ("La dérivée du polynôme est :", derivation(p1))
```

Listing 2.10 – Opérations sur les polynômes

Conclusion

À l'issue de ce travail de maturité, j'ai pu observer l'évolution de ma réflexion. Ma manière d'aborder un problème a changé ainsi que la compréhension théorique des polynômes jusqu'à la réalisation d'un programme informatique capable de les manipuler efficacement. Cela m'a permis de prendre conscience de la rigueur nécessaire et de l'importance de considérer toutes les situations possibles lors de la création de programmes informatiques et de traiter même les cas qui paraissent rares ou secondaires comme les polynômes nuls ou les équations à discriminant négatif.

J'ai commencé mon travail avec la réalisation de programmes permettant des opérations fondamentales comme l'addition et la soustraction de polynômes. Cela m'a permis de poser un cadre de compréhension, de développer une démarche logique, comprendre qu'il était important de structurer les données ainsi qu'anticiper les erreurs potentielles.

Individuellement, la fonction d'addition m'a appris à faire en sorte que les données soient structurées pour que le polynôme le plus long soit toujours prioritaire alors que la soustraction m'a appris à considérer des coefficients nuls, afin d'égaliser la longueur des polynômes et d'éviter des incohérences dans les résultats.

C'est grâce à ces premières étapes que j'ai pu réaliser les limites de la logique humaine face à la logique des machines et appris à penser comme un programme le ferait avec une succession d'étape et un détail de chaque opération. Quant à elle, la multiplication de polynômes a été une difficulté et un défi. La multiplication a besoin de prendre en compte la distribution de chaque terme, puis additionne les coefficients de même degré. Reproduire ces démarches de manière algorithmique m'a demandé de comprendre la correspondance entre les listes et les degrés des monômes. Ma réflexion m'a démontré par la suite qu'une conception claire et systématique était d'une grande importance. Chaque détail compte pour que le résultat final soit correct et fidèle à ce qui aurait été obtenu manuellement.

L'implémentation du schéma de Horner m'a apporté une maîtrise différente des polynômes. L'algorithme de ce schéma est plus répétitif et plus systématique, ce qui m'a demandé de comprendre non seulement la mécanique de la division euclidienne par $(x - a)$ mais aussi la manière dont une fonction peut automatiser de manière fiable un processus qui se répète.

La résolution des équations quadratiques avec la prise en compte des solutions réelles et complexes m'a permis de comprendre concrètement le rôle du discriminant et par la suite d'adapter le code pour couvrir toutes les situations. Il était nécessaire d'avoir une analyse préalable et de comprendre comment manuellement le problème était représenté avant de le traduire en algorithme.

L'assemblage final des différentes fonctions en un programme complet a représenté un travail d'analyse. Il a fallu réfléchir à la logique et à l'organisation des fonctions. Ensuite, j'ai du programmer un menu principal clair pour l'utilisateur. Cela m'a demandé de la

rigueur dans la structure et dans la lisibilité car chaque fonction est indépendante mais s'intègre dans un système global.

En outre, le premier chapitre consacré à l'histoire des polynômes m'a fait comprendre l'importance et l'étendue historique de ce concept. Les Babyloniens, les Grecs, Al-Khwarizmi et les mathématiciens de la Renaissance ont tous contribué au développement des équations polynomiales et à l'histoire des mathématiques.

Ainsi, les équations polynomiales témoignent de l'évolution de la pensée mathématique. Avoir retracé en partie cette évolution, des raisonnements géométriques jusqu'à la formalisation algébrique, m'a permis de voir la continuité entre les premières méthodes et la logique moderne. J'ai pu relier l'histoire des mathématiques à la programmation actuelle. Depuis toujours, la rigueur, la logique et les méthodes restent essentielles.

En guise de conclusion, je pourrais dire que ce travail de maturité m'a permis de développer des compétences essentielles : la traduction d'un concept en algorithme, la rigueur analytique et la capacité de structurer des éléments et de relier l'histoire à la pratique. Il m'a permis d'avoir une maîtrise de la programmation qui s'est approfondie et une compréhension des polynômes et de leurs applications. Cette expérience m'a non seulement enrichie sur le plan technique, mais m'a également appris à organiser un projet méthodiquement et à anticiper les problèmes avant même qu'ils ne surviennent, des compétences qui me seront précieuses pour mes futurs projets.

Bibliographie

Livres

Bachmann, Marcel Yves ; Cattin, Huber ; Epiney, Paul ; Haeberli, François ; Jenny, Gérard. 1992/2004. *Méthodes numériques*. [Ouvrage] Genève : Editions du Tricorne : 1992/2004

Shovic, John ; Simpson, Alan. 2021. *Python Tout-en-un*. [Ouvrage] Paris : Éditions First, 2021.

Ressources électroniques

Khan Academy. 2025. *Algebra – Polynomials*. [En ligne] https://www.khanacademy.org/math/algebra-home

Mazliak, Laurent. 2015. *Euclide*. [PDF] Paris : Université Paris, LPSM, 2015. https://perso.lpsm.paris/~mazliak/Euclide.pdf

Testbook. 2023. *Father of Polynomial – History, Definition and Mathematicians*. [En ligne] 2023. https://testbook.com/maths/father-of-polynomial

Wikipedia. s.d. *Niccolò Fontana Tartaglia*. [En ligne] https://fr.wikipedia.org/wiki/Niccol%C3%B2_Fontana_Tartaglia

Wikipedia. s.d. *Équation diophantienne*. [En ligne] https://fr.wikipedia.org/wiki/%C3%89quation_diophantienne

Wikipedia. s.d. *Méthode de Cardan*. [En ligne] https://fr.wikipedia.org/wiki/M%C3%A9thode_de_Cardan

Wikipedia. s.d. *Al-Khwarizmi*. [En ligne] https://fr.wikipedia.org/wiki/Al-KhwÃćrizmÃő

Wikipedia. s.d. *Diophante d'Alexandrie*. [En ligne] https://fr.wikipedia.org/wiki/Diophante_d%27Alexandrie

Wikipedia. s.d. *Euclide*. [En ligne] https://fr.wikipedia.org/wiki/Euclide

Wikipedia. s.d. *Scipione del Ferro* . [En ligne] https://fr.wikipedia.org/wiki/Scipione_del_Ferro

Bibmath. 2023. *Ludovico Ferrari*. [En ligne] https://www.bibmath.net/bios/index.php?action=affiche&quoi=ferrari

Wikipedia. s.d. *Gerolamo Cardano*. [En ligne] https://en.wikipedia.org/wiki/Gerolamo_Cardano

Friberg, Jöran. 2003. *On Ancient Babylonian Algebra and Geometry*. [PDF] Resonance, 2003. https://www.ias.ac.in/article/fulltext/reso/008/08/0027-0042

Table des illustrations

1 Illustration de code informatique (page de garde) – Source : https://www.phyto-info.com/fr/blog/langage-machine/ 1

1.1 Portrait d'Euclide, mathématicien grec antique 8
Source : https://www.britannica.com/biography/Euclid-Greek-mathematician, *"Euclid"* 8
1.2 Portrait de Diophante d'Alexandrie, mathématicien grec 9
Source : https://fr.wikipedia.org/wiki/Diophante_d%27Alexandrie, *"Diophante d'Alexandrie"* 9
1.3 Portrait d'Al-Khwarizmi, mathématicien perse 10
Source : https://science4fun.info/al-khwarizmi/, *"Al-Khwarizmi"* 10

ANNEXES

Au cours de mon travail, j'ai exploré brièvement le fonctionnement interne des calculs informatiques et leur lien avec la logique binaire. J'ai donc réalisé un algorithme qui convertit un nombre entier en binaire. J'ai pu comprendre comment les ordinateurs représentent et manipulent les nombres et les données à l'aide du système binaire.

Cependant, l'optimisation des calculs s'est avérée très complexe et peu cohérente avec l'objectif principal de mon travail, c'est pourquoi j'ai intégré cette partie en annexe. De plus, elle illustre ma démarche de recherche et mon intérêt pour la logique informatique.

Les systèmes numériques

Les systèmes numériques sont la base de l'informatique moderne avec un système de numérotation qui repose essentiellement sur le code binaire qui est un système utilisant uniquement deux symboles 0 et 1.

Grâce à ce système il est possible de représenter toutes les informations manipulées par un ordinateur. Peu importe leur nature : il peut s'agir de nombre de lettres, d'image ou bien encore de sons. Par rapport à un système standard nommé ASCII, la lettre «B» peut-être représentée en binaire par la séquence 1000010. Le code binaire permet de réaliser des opérations logiques et arithmétiques fondamentales comme la soustraction, la division, la multiplication et peut aussi représenter des fonctions complexes qui peuvent être utilisées dans les programmes informatiques.

Toute tâche informatique repose sur des manipulations en binaire. Ce système est un pont entre la logique humaine et la logique machine. L'étudier montre son importance et permet de réaliser à quel point les calculs et la représentation des données sont précis et efficaces.

Création d'un algorithme de conversion de nombre binaire à décimal

La démarche que j'ai utilisée pour la création de mon algorithme de conversion d'un nombre décimal en un nombre binaire a d'abord été analytique. Il fallait que je comprenne le principe mathématique de la conversion. Je me suis donc appuyée sur le fait que tous les nombres entiers positif peuvent se décomposer en somme de puissances de deux.

La première étape est de déterminer la plus grande puissance de deux inférieure ou égale au nombre à convertir. C'est ce qui m'a permis de savoir où placer le premier 1 dans la représentation binaire. Pour cela, j'ai créé la fonction `pgp2(nombre)` qui calcule cette puissance de manière itérative. Puis dans la fonction `entierabinaire(nombre)`, j'ai implémenté la logique qui permet de construire progressivement la chaîne binaire. À chaque étape, on soustrait la puissance de deux correspondante du nombre et on ajoute 1 dans la chaîne, tout en insérant les 0 nécessaires pour les puissances absentes.

Lors de ce travail, j'ai rencontré plusieurs difficultés et obstacles. Par exemple, une difficulté a été de gérer correctement les puissances manquantes entre deux puissances de 2 et d'insérer des 0 entre, ainsi que de calculer combien il fallait en insérer. C'est ce qui m'a

demandé de manipuler les indices et de vérifier attentivement chaque étape de la logique.

De plus, j'ai rencontré le cas où le nombre initial était une puissance exacte de 2, il fallait donc ne pas insérer de zéro qui n'était pas nécessaire et éviter les erreurs d'indice.

Il fallait aussi s'assurer que le programme fonctionne pour tous les nombres entiers positifs, qu'ils soient très grands ou proches 0 et c'est face à ces difficultés que j'ai réussi à structurer mon code de manière claire avec des boucles et d'incrémenter des compteurs logiques.

L'algorithme final est capable de convertir n'importe quel entier positif dans sa représentation binaire correcte, tout en illustrant les étapes de raisonnement qui m'ont guidée dans sa conception.

```
entier = int(input("Entrez un nombre entier positif : "))

# la fonction calcule la plus grande puissance de 2 inférieure ou égale au nombre donné
def pgp2(nombre):
    puissance = 0 # compteur de la puissance (exposant)
    valeur = 1    # valeur correspondante à 2^puissance (commence à 2^0 = 1)

    # tant que la prochaine puissance de 2 (valeur * 2) est inférieure ou égale au nombre, on augmente la puissance
    while valeur * 2 <= nombre:
        valeur = valeur * 2  # on multiplie la valeur par 2 (passe à la puissance suivante)
        puissance = puissance + 1 # on incrémente l'exposant

    # retourne la plus grande puissance d'exposant trouvée
    return puissance

def entier_a_binaire(nombre):
    if nombre == 0:
    print ("0 a pour représentation binaire : 0")
    return

    resultat = ""   # chaîne vide qui va contenir la représentation binaire
    p_initial = pgp2(nombre)
    p = p_initial

    #tant que le nombre n'a pas été réduit à zéro
    while nombre > 0:
        # calcule la valeur 2^p
        puissance2 = 1
        compteur = 0
        while compteur < p:
            puissance2 = puissance2 * 2
            compteur += 1

        # ajoute un "1" dans le résultat car cette puissance est pré sente
        resultat = resultat + "1"
```

```
36
37          # soustrait cette puissance de 2 au nombre pour continuer
38          nombre = nombre - puissance2
39          p_ = p
40
41          if nombre == 0:
42          break
43
44          p = pgp2(nombre)
45
46          # ajoute des "0" pour les puissances inférieures manquantes
    jusqu'à atteindre la position p
47          diff = 0
48          for i in range(p_ - p - 1):
49                  resultat = resultat + "0"
50                  diff = p_initial - len(resultat) + 1
51
52      if diff > 0 :
53          while diff:
54              resultat = resultat + "0"
55              diff -= 1
56
57      print(entier, "a pour représentation binaire :", resultat)
58
59  # appel de la fonction avec le nombre entré par l'utilisateur
60  entier_a_binaire(entier)
```

Listing 2.11 – Conversion d'un nombre entier en sa représentation binaire